

Análise Comparativa de LLMs Open-Source para a Conversão de Normas de Arquitetura, Engenharia e Construção para o Formato RASE

Eike Natan Sousa Brito^{a,*}, André Carvalho^a and Marco Antônio Brasiel Sampaio^a

^aUniversidade Federal de Sergipe, Cidade Universitária Prof. José Aloísio de Campos, São Cristóvão, 49100-000, Sergipe, Brasil

ARTICLE INFO

Palavras-chave:

Aprendizado de Máquina
Processamento de Linguagem Natural
LLM
Engenharia de Prompt
BIM
RASE

Resumo

A metodologia *Building Information Modeling* (BIM) organiza o planejamento e a execução de obras em torno de um modelo tridimensional detalhado, reduzindo retrabalho e antecipando interferências antes do início da construção. Esse modelo, porém, não resolve sozinho a verificação da conformidade dos projetos com as normas técnicas regulatórias, cuja natureza interpretativa torna a automação um problema em aberto. A metodologia RASE (*Requirement, Applicability, Selection, Exception*) endereça essa lacuna ao reescrever cada trecho normativo como dado computável, permitindo que *softwares* BIM verifiquem a conformidade automaticamente; a conversão das normas para esse formato, contudo, ainda é feita manualmente, regra a regra, o que se torna um gargalo para a automação. O Processamento de Linguagem Natural, em particular os Modelos de Linguagem de Grande Porte (LLMs), surge como caminho promissor para automatizar essa conversão, podendo ser adaptado à tarefa por meio da Engenharia de Prompt, técnica leve que ajusta apenas a instrução enviada ao modelo. Este trabalho avalia comparativamente seis LLMs *open-source* (Llama, Dolphin, Gemma, Mistral, Alpaca e Qwen) na conversão automática de normas técnicas brasileiras da AEC para o formato RASE, utilizando exclusivamente Engenharia de Prompt em três níveis sucessivos de normalização (N1: segmentação atômica das sentenças normativas; N2: identificação dos operadores RASE; N3: estruturação semântica em JSON). A comparação foi conduzida em cinco experimentos (EN1, EN2, EN1N2, EN3 e EN1N2N3) sobre um *dataset* de 79 normas anotadas da NBR 9050, com treze métricas distribuídas em cinco famílias (lexicais, semânticas, de distância, voltadas para geração e de classificação) e os modelos servidos localmente via Ollama, sob protocolo idêntico de *dataset*, *prompts* e *hardware*. Os resultados mostram que a decomposição em três níveis, apoiada apenas em Engenharia de Prompt, é suficiente para gerar representações RASE válidas, alcançando similaridade semântica próxima de 0,90 nos melhores cenários; o Llama obteve a melhor média global (0,730), liderando as etapas do nível N3 (EN3 e EN1N2N3), enquanto o Dolphin apresentou a melhor relação custo-benefício, cerca de noventa vezes mais rápido no *pipeline* encadeado.

1. Introdução

Projetar e revisar projetos de engenharia civil é uma atividade intensiva em conhecimento (*knowledge-intensive activity*): exige que engenheiros interpretem normas, conciliem informações heterogêneas e tomem decisões fundamentadas em *expertise* acumulada ao longo de anos de prática. A *Engineering Informatics* estuda justamente como métodos computacionais podem apoiar essas atividades, transformando o conhecimento contido em documentos, modelos e normas em uma forma processável por máquina. Dentro desse campo, automatizar a verificação de projetos de edificações contra normas técnicas regulatórias é um problema antigo e ainda em aberto, já que essas normas são escritas em linguagem natural e exigem interpretação. O presente trabalho pode ser caracterizado como um problema dessa área: usa Inteligência Artificial (IA) para converter normas técnicas em uma representação computável que ferramentas *Building Information Modeling* (BIM) possam verificar automaticamente, tarefa que se situa na interseção entre representação de conhecimento, Processamento de Linguagem Natural e a prática de engenharia.

Na engenharia civil brasileira, projetar e fiscalizar obras ainda envolve uma enorme quantidade de retrabalho. Parte desse desperdício vem da forma como os projetos são representados: o desenho 2D, padrão histórico do setor, esconde colisões e omite informações que só aparecem no canteiro. A migração para modelos tridimensionais e, mais

*Autor correspondente

✉ eike.sousa@hotmail.com (E.N.S. Brito); andre@comp.ufs.br (A. Carvalho); marco.sampaio@ufs.br (M.A.B. Sampaio)
ORCID(s):

especificamente, para a metodologia *Building Information Modeling* (BIM) surgiu como resposta a esse problema (Eastman, Teicholz and Sacks, 2011).

O BIM organiza o planejamento e a execução da obra em torno de um modelo 3D detalhado. A metodologia divide-se em dimensões que vão do 3D ao 7D: 3D para o modelo geométrico; 4D para a programação temporal; 5D para custos; 6D para sustentabilidade; e 7D para a gestão da operação pós-construção (Eastman et al., 2011). BIM não é um *software*, e sim um conceito apoiado em sistemas informatizados como o CAD, implementado por ferramentas como o Revit (Autodesk) e o ArchiCAD (Graphisoft).

Modelar em BIM permite visualizar interferências antes do início da obra, reduzindo o retrabalho em campo. As aplicações típicas vão da análise de projetos ao orçamento, passando pelo planejamento, pela coordenação modular e pela gestão do ciclo de vida (Eastman et al., 2011).

O BIM, porém, não resolve sozinho a verificação dos projetos contra as normas regulatórias. As normas regulatórias (ou técnicas) são documentos que estabelecem requisitos, critérios e procedimentos obrigatórios para assegurar a segurança, a qualidade e o desempenho das edificações; no Brasil, são editadas pela Associação Brasileira de Normas Técnicas (ABNT) na forma das NBRs, como a NBR 9050, voltada à acessibilidade. Cada jurisdição mantém seu próprio conjunto de regras, em geral redigidas em linguagem natural e de caráter interpretativo, o que torna a verificação automática de conformidade um problema em aberto (Dimyadi and Amor, 2013; Solihin and Eastman, 2015).

Tornar essas normas verificáveis por máquina exige convertê-las em uma representação computável. Com essa finalidade, Hjelseth and Nisbet (2011) propuseram a metodologia RASE, que reescreve cada trecho normativo como dado estruturado, decomposto em quatro atributos: **Requisito** (o que precisa ser atendido), **Aplicabilidade** (onde a regra se aplica), **Seleção** (opções para cumpri-la) e **Exceção** (casos em que a regra não vale). Uma vez organizada em RASE, a norma viabiliza ferramentas integradas a *softwares* BIM, como o Revit, capazes de validar o projeto elemento a elemento (Santos and Sampaio, 2021). O gargalo está antes disso: a conversão da norma para RASE ainda é feita manualmente, regra a regra, e, como um modelo BIM pode reunir centenas ou milhares de elementos sujeitos a normas, o custo desse trabalho cresce rapidamente.

Automatizar essa conversão exige reconhecer padrões em textos em linguagem natural, tarefa para a qual o Aprendizado de Máquina (AM) se mostra promissor. A motivação é reforçada pelo próprio BIM: quanto maior a adoção, maior o volume de dados gerado por cada projeto, o que atrai cientistas de dados, profissionais cuja função é transformar grandes volumes de informação em valor para as organizações (Davenport and Patil, 2012). O AM, subárea da IA, estuda como construir algoritmos capazes de aprender padrões diretamente dos dados, sem programação explícita das regras (Tan, Steinbach and Kumar, 2006); a partir desses padrões, os modelos fazem previsões e respondem a situações novas. Na engenharia civil, a IA já automatiza etapas que antes dependiam de inspeção humana, da detecção de patologias em estruturas à previsão de prazos de obra (Rane, Choudhary and Rane, 2023; Liu, Zhang and Li, 2022).

Três exemplos recentes ilustram esse movimento. Liu et al. (2022) combinaram BIM, AM e regras de *design* para otimizar o processo de montagem da construção, com ganhos em eficiência e custo. Alawadhi and Yan (2023) treinaram Redes Neurais Artificiais sobre dados BIM para reconhecer objetos de construção em fotos: imagens sintéticas geradas a partir do modelo produziram segmentação semântica eficaz em fotografias reais, abrindo caminho para o uso de dados sintéticos em problemas arquitetônicos. Em outra frente, Saka, Taiwo, Saka, Oluleye, Dauda and Akanbi (2024) alimentaram um modelo com 2.280 projetos de demolição para prever a circularidade de resíduos: o sistema estima as quantidades de material reciclável e destina os resíduos ao aterro adequado, apoiando o planejamento.

Quando o objeto de análise é o próprio texto normativo, a automação recai sobre o Processamento de Linguagem Natural (PLN), subárea da IA dedicada ao tratamento computacional da linguagem humana em tarefas como tradução, sumarização e classificação de texto. Dentro do PLN, a IA Generativa reúne os modelos capazes de produzir novos conteúdos (texto, imagem ou áudio) a partir do que aprenderam no treinamento. Entre suas manifestações mais relevantes estão os Modelos de Linguagem de Grande Porte (LLMs) (Zhao, Zhou, Li, Tang, Wang, Hou, Min, Zhang, Zhang, Dong, Du, Yang, Chen, Chen, Jiang, Ren, Li, Tang, Liu, Nie and Wen, 2023), categoria que inclui o ChatGPT, da OpenAI (OpenAI, 2024), e o Llama 3, da Meta (AI@Meta, 2024).

Os LLMs são sistemas de IA voltados a entender e produzir linguagem humana em escala. O treinamento envolve volumes massivos de texto, o que permite ao modelo capturar nuances linguísticas e contextuais. Quase todos os LLMs atuais usam alguma variação da arquitetura *transformer*, escolhida por sua eficiência no processamento de sequências longas (Zhao et al., 2023).

A adaptação desses modelos a domínios específicos pode seguir caminhos diferentes. O mais leve e acessível é a Engenharia de Prompt (EP): em vez de retrainar pesos ou montar uma infraestrutura de recuperação, ajusta-se apenas a instrução enviada ao modelo.

No setor AEC, os LLMs podem auxiliar tanto na leitura das normas quanto na sua conversão para formatos processáveis por máquina (Fuchs, Witbrock, Dimyadi and Amor, 2024). A literatura, porém, ainda tem lacunas: uma delas é justamente a interpretação de leis, códigos e padrões regulatórios, cuja complexidade semântica historicamente desafiou os métodos de IA anteriores (Fuchs et al., 2024).

Diante dessa lacuna, este trabalho decompõe a conversão de normas para RASE em três níveis sucessivos de normalização, apoiados unicamente em Engenharia de Prompt: o nível **N1** segmenta a sentença normativa em regras atômicas; o **N2** identifica, em cada regra, os operadores RASE; e o **N3** estrutura o resultado em JSON. Sobre essa decomposição, comparam-se seis LLMs *open-source* (Llama, Dolphin, Gemma, Mistral, Alpaca e Qwen), servidos localmente, em cinco experimentos que avaliam cada nível isoladamente e em *pipeline* encadeado. A qualidade das saídas é aferida por nove métricas de similaridade textual em quatro famílias: lexicais (FuzzyWuzzy e TF-IDF), semânticas (SBERT, BERTimbau e Multilingual), de distância (*Word Mover's Distance* com FastText e NILC) e voltadas para geração (BERTScore e ROUGE-L); a essas somam-se quatro métricas de classificação (Acurácia, Precisão, Revocação e F1-score).

As contribuições deste trabalho situam-se na interseção entre Arquitetura, Engenharia e Construção (AEC) e Computação: para a AEC, aplica modelos de PLN à checagem de normas de construção e propõe níveis sucessivos de normalização sobre a metodologia RASE de Hjelseth and Nisbet (2011); para a Computação, leva LLMs consolidados na literatura a um novo domínio e os submete a uma avaliação sistemática de desempenho.

2. Background

2.1. Normas de Engenharia e a Metodologia RASE

As normas técnicas formam o arcabouço regulatório do setor AEC: definem os requisitos mínimos de segurança, desempenho, qualidade e interoperabilidade dos projetos. No plano internacional, a Organização Internacional de Padronização (ISO) publica normas adotadas por diversos países; em BIM, a referência consolidada é a série ISO 19650, voltada à organização e à digitalização da informação sobre obras de construção (iso, 2018). Independentemente da jurisdição, cada norma combina três camadas textuais: (i) definições e referências, sem valor de verificação direta; (ii) requisitos prescritivos, com limites numéricos e geométricos passíveis de checagem automática; e (iii) recomendações e exceções, em texto livre, frequentemente condicionadas a categorias específicas do edifício. Em qualquer das três camadas o texto bruto é longo, recursivo e fortemente dependente de definições prévias, o que torna a tradução para regras computáveis um trabalho não trivial e, hoje, predominantemente manual.

A aplicação dessas normas permanece, em grande parte, manual: o profissional lê trechos longos de texto, identifica os requisitos pertinentes e verifica item a item contra o projeto. Em um modelo BIM com centenas ou milhares de elementos sujeitos a normas simultâneas, esse esforço cresce de forma combinatória; Eastman, Lee, Jeong and Lee (2009) mostram que a verificação manual é cara, frágil e pouco escalável, motivando a pesquisa em checagem automática baseada em regras. Automatizá-la, porém, esbarra em três obstáculos: a ambiguidade e as referências cruzadas da linguagem natural em que as normas são escritas (Dimyadi and Amor, 2013); a variação regional, que limita o reaproveitamento do que já foi formalizado em outros contextos (Dimyadi and Amor, 2013; Solihin and Eastman, 2015); e a dificuldade semântica de distinguir, dentro de um trecho, o que é requisito, aplicabilidade e exceção, o que exige um esquema padronizado e interoperável entre ferramentas (Solihin and Eastman, 2015).

A resposta que a literatura vem consolidando é estruturar a norma em um esquema computável antes da verificação. Beach, Rezgui, Li and Kasim (2015) demonstram que um sistema baseado em regras automatiza a checagem regulatória da construção quando as normas estão formalizadas. Nessa direção, Hjelseth (2015) propôs duas metodologias complementares: a Tx3 e a RASE. A Tx3 classifica cada informação do regulamento em **Transcrever** (instruções diretamente transcritíveis em regras computáveis), **Transformar** (instruções que precisam ser reescritas preservando o sentido) e **Transferir** (requisitos imprecisos, que exigem análise de um especialista). Os elementos transcritíveis e transformáveis são reescritos e submetidos à RASE, que marca cada trecho normativo em quatro operadores lógicos (Hjelseth and Nisbet, 2011): **Requisito** (o dever associado, em geral expresso pelo imperativo “deve”), **Aplicabilidade** (a quem ou ao que o requisito se aplica), **Seleção** (um subconjunto da aplicabilidade) e **Exceção** (os casos excluídos da regra). Cada operador recebe ainda atributos estruturados, como objeto, propriedade, comparador, valor e unidade, o que aproxima o texto de uma forma diretamente verificável. Por exemplo, na sentença “a inclinação transversal deve

ser de até 2% para pisos internos”, a aplicabilidade é “pisos”, a seleção “internos” e o requisito “inclinação transversal menor ou igual a 2%”. Uma vez organizada em RASE, a norma viabiliza ferramentas integradas a *softwares* BIM, como o Revit, capazes de validar o projeto elemento a elemento (Santos and Sampaio, 2021). Aplicar a RASE manualmente, contudo, reintroduz o gargalo que se quer eliminar: marcar operador a operador exige um especialista e não escala para o volume de normas do setor. No fundo, identificar Requisito, Aplicabilidade, Seleção e Exceção em uma sentença é um problema de classificação e de estruturação de texto, exatamente o tipo de tarefa que técnicas de Aprendizado de Máquina aprendem a partir de exemplos.

2.2. Modelos de Linguagem de Grande Porte e Métricas de Validação Textual

Modelos de Linguagem de Grande Porte (LLMs) são modelos de IA que combinam Aprendizado de Máquina, redes neurais profundas e técnicas de Processamento de Linguagem Natural para entender e gerar linguagem humana (Touvron, Lavril, Izacard, Martinet, Lachaux, Lacroix, Rozière, Goyal, Hambro, Azhar, Rodriguez, Joulin, Grave and Lample, 2023). São tipicamente pré-treinados de forma não supervisionada sobre bilhões de palavras, aprendendo a prever a estrutura das sentenças por probabilidade, e adotam a arquitetura *Transformer* (Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser and Polosukhin, 2017), eficiente no tratamento de dependências contextuais em sequências longas. O lançamento do ChatGPT e do GPT-4 (OpenAI, 2024) popularizou esses modelos, e, como resposta a modelos fechados, surgiram alternativas de pesos abertos como o LLaMA (Touvron et al., 2023; AI@Meta, 2024). Entre as técnicas que especializam um LLM para uma tarefa de domínio sem alterar seus pesos, a Engenharia de Prompt é a mais leve: consiste em desenhar com cuidado a instrução enviada ao modelo, prática essencial para extrair respostas adequadas ao contexto (Brown, Mann, Ryder, Subbiah et al., 2020).

Avaliar a saída de um LLM em tarefas de conversão de texto exige compará-la a uma referência. Como a correspondência exata raramente é desejável, já que uma paráfrase correta também deve contar como acerto, emprega-se um conjunto de métricas de similaridade, organizadas em quatro famílias (Jurafsky and Martin, 2009). As **lexicais** comparam a forma de superfície das *strings*: a *FuzzyWuzzy*, baseada na distância de Levenshtein, e a similaridade do cosseno sobre vetores TF-IDF (Manning, Raghavan and Schütze, 2008). As **semânticas** apoiam-se em *sentence embeddings* que aproximam textos de sentido equivalente, como o *Sentence-BERT* (Reimers and Gurevych, 2019), o BERTimbau (especializado em português) (Souza, Nogueira and Lotufo, 2020) e modelos multilíngues. As **de distância** medem o custo de alinhar os *embeddings* das palavras de um texto aos do outro, caso da *Word Mover's Distance* (Kusner, Sun, Kolkin and Weinberger, 2015), executável sobre *embeddings* do FastText (Bojanowski, Grave, Joulin and Mikolov, 2017) ou do projeto NILC. As **voltadas para geração**, originárias da tradução automática e da sumarização, medem o quanto a saída reproduz a referência: o *BERTScore* (Zhang, Kishore, Wu, Weinberger and Artzi, 2020), que alinha *tokens* por similaridade contextual, e o *ROUGE-L* (Lin, 2004), baseado na maior subsequência comum. A essas somam-se métricas de classificação (acurácia, precisão, revocação e *F1-score*), derivadas da matriz de confusão, que tornam explícitas omissões e alucinações ao tratar a avaliação como um problema de extração de informação (Manning et al., 2008).

2.3. Trabalhos Relacionados

A integração do BIM com tecnologias de IA generativa ainda é pouco explorada no setor AEC, mas é objeto de uma onda recente de trabalhos. Rane et al. (2023) analisam a integração entre BIM e modelos como ChatGPT e Bard, identificando ganhos em *design* e tomada de decisão, mas apontando obstáculos persistentes (falta de padronização dos dados, deficiências de segurança e dificuldade dos modelos em interpretar a semântica de dados BIM) e propondo um *framework* de adoção. Lin (2023) apresentam o JULE, um *chatbot* que combina BIM e PLN para entregar, no canteiro, consultas em tempo real sobre dimensões de componentes sem abrir as interfaces convencionais, com alta precisão na identificação de intenção, mas condicionado a dados BIM bem estruturados. Sammour, Xu, Wang, Hu and Zhang (2024) aplicam GPT-3.5 e GPT-4 a 385 questões de segurança e três técnicas de Engenharia de Prompt (*Prompt Direct*, *Chain-of-Thought* e *Few-Shot*): o GPT-4 atingiu 84,6% de acerto contra 73,8% do GPT-3.5, mas ambos falharam em questões de emergência e cálculo, expondo a fragilidade da confiabilidade técnica. Por fim, Samsami (2024) mostram que *prompts* genéricos não funcionam no setor AEC e que *prompts* específicos reduzem erros, embora alucinações em tarefas complexas e a necessidade de validação externa persistam. No conjunto, os trabalhos confirmam o potencial dos LLMs no setor, mas revelam uma lacuna comum: a ausência de uma representação estruturada e formal que traduza o conteúdo normativo em regras computáveis. Sem ela, os LLMs extraem texto e geram *insights*, mas não convertem a norma em algo diretamente verificável por máquina, exatamente o ponto que este trabalho procura endereçar ao acoplar os LLMs à metodologia RASE.

3. RASE-PE: Conversão de Normas para RASE com Engenharia de Prompt

A metodologia proposta, aqui denominada **RASE-PE**, sustenta-se em três decisões: (i) decompor a conversão de normas para o formato RASE em três níveis sucessivos de normalização (N1, N2 e N3); (ii) usar a Engenharia de Prompt como única técnica de especialização dos modelos, sem alterar pesos nem infraestrutura de recuperação; e (iii) comparar sistematicamente seis LLMs *open-source* servidos localmente via Ollama, sob protocolo idêntico de *dataset*, *prompts* e *hardware*. As subseções a seguir detalham a decomposição em três níveis, a Engenharia de Prompt empregada, os modelos avaliados, o *dataset*, as métricas de validação e, por fim, o ambiente computacional e a estrutura do código.

3.1. Normalização em Três Níveis

Sair do texto bruto de uma norma e chegar à representação RASE em JSON exige três raciocínios distintos: segmentação sintática, classificação semântica e extração de atributos estruturados. Pedir tudo em uma única chamada ao LLM tende a degradar a saída, com alucinações, omissões e JSONs inválidos. O RASE-PE decompõe a tarefa em três níveis sucessivos, isolando cada decisão e permitindo validar cada etapa em separado:

- **N1: Segmentação Atômica.** Sentenças normativas costumam embutir mais de uma regra coordenada (por exemplo, “a inclinação transversal deve ser de até 2% para pisos internos e de até 3% para pisos externos” contém duas regras). O N1 quebra a sentença original em itens atômicos, com uma única regra por item, condição para que as etapas seguintes atribuam corretamente os operadores RASE.
- **N2: Identificação dos Operadores RASE.** A partir de cada regra atômica, o N2 identifica quais trechos correspondem a Requisito, Aplicabilidade, Seleção e Exceção. É o núcleo da formalização: sem ele, o texto permanece em linguagem natural.
- **N3: Estruturação Semântica em JSON.** Classificar um trecho como R, A, S ou E ainda o deixa em linguagem natural; para a verificação automática contra um modelo BIM é preciso extrair os dados para campos estruturados. O N3 produz um JSON com os campos *type*, *object*, *property*, *comparison*, *target* e *unit*.

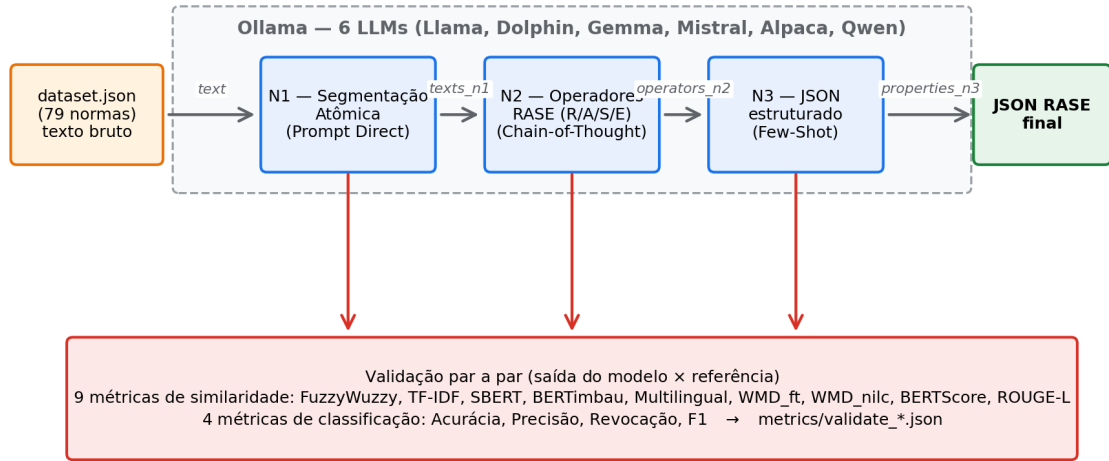
O encadeamento N1 → N2 → N3 forma o *pipeline* completo do RASE-PE (Figura 1), do texto bruto à representação RASE estruturada. Para isolar a contribuição de cada nível e medir a propagação de erros ao longo da cadeia, a avaliação é organizada em cinco experimentos, apresentados na Seção 4.

3.2. Engenharia de Prompt

A especialização dos LLMs foi feita exclusivamente por Engenharia de Prompt: toda a adaptação à tarefa vive nos *prompts*, organizados por nível e por operador. Três técnicas clássicas foram aplicadas, cada uma escolhida pela natureza do nível em que atua. O N1 usa *Prompt Direct*, uma instrução direta e objetiva, adequada a uma tarefa estruturalmente simples (segmentar a sentença em sub-regras) e econômica em *tokens* e latência. O N2 usa *Chain-of-Thought*, pedindo ao modelo que explicita o raciocínio em etapas antes de responder, o que reduz alucinações ao classificar trechos com múltiplos operadores. O N3 usa *Few-Shot*, embutindo exemplos completos de entrada e saída que ancoram o esquema JSON rígido; como cada operador RASE tem natureza distinta, adotou-se um *prompt* por operador (Aplicabilidade, Requisito, Seleção e Exceção), cada um com o esquema de saída, heurísticas de mapeamento por padrões textuais comuns (por exemplo, “uso público” → *property* “uso”, *comparison* “=”, *target* “público”) e exemplos próprios.

3.3. Modelos LLM Avaliados

Foram avaliados seis modelos *open-source*, executados localmente via Ollama (Ollama, 2024), escolhidos por apresentarem perfis complementares de peso, alinhamento, idioma e foco de *fine-tuning* (Tabela 1). Todos compartilham a arquitetura *Transformer* decodificadora e o paradigma de pré-treinamento massivo seguido de pós-treinamento por instruções, mas diferem em dados, alinhamento e escala. Salvo o Llama, avaliado em sua versão oficial multilíngue de propósito geral, todos foram executados a partir de *checkpoints* adaptados ao português brasileiro publicados por terceiros e disponíveis no registro do Ollama. Executar os seis modelos no mesmo *hardware*, com os mesmos *prompts* e o mesmo *dataset*, é o que garante a comparabilidade entre os resultados.

Pipeline de conversão de normas técnicas para o formato RASE**Figura 1:** Pipeline RASE-PE de conversão de normas técnicas para o formato RASE.**Tabela 1**Modelos LLM *open-source* avaliados, com origem e *tag* servida via Ollama.

Modelo	Origem	Tag servida via Ollama	Parâmetros
Llama	Meta	llama3.1:8b	8B
Dolphin	cnmoro (LLaMA-3 PT)	cnmoro/llama-3-8b-dolphin-portugues e-v0.3:4_k_m	8B (Q4_K_M)
Gemma	brunoconterato (PT-BR)	brunoconterato/Gemma-3-Gaia-PT-BR-4 b-it:f16	4B (FP16)
Mistral	cnmoro (PT)	cnmoro/mistral_7b_portuguese:q4_K_M	7B (Q4_K_M)
Alpaca	splitpierre (Bode PT-BR)	splitpierre/bode-alpaca-pt-br:13b-Q 4_0	13B (Q4_0)
Qwen	cnmoro (PT)	cnmoro/Qwen2.5-0.5B-Portuguese-v1:q 4_k_m	0,5B (Q4_K_M)

3.4. Dataset

O *dataset* foi consolidado em um único arquivo JSON com **79 normas anotadas**, oriundas de regulamentações brasileiras da AEC, com ênfase na NBR 9050 (abn, 2020), adaptadas a partir do trabalho de Mendonça and Manzione (2020). A NBR 9050 foi escolhida por seu perfil majoritariamente prescritivo, com valores numéricos diretamente verificáveis em um modelo BIM e exceções bem delimitadas. Cada entrada reúne a decomposição RASE de referência nos três níveis: o texto bruto da norma (*text*); a lista de regras atômicas (*texts_n1*); para cada regra, os quatro operadores RASE (*operators_n2*, com as chaves *requirements*, *applicability*, *selection* e *exception*); e, para cada operador, o trecho classificado (*text_n2*) e as propriedades estruturadas em JSON (*properties_n3*). Operadores ausentes preservam a estrutura com *strings* vazias. O mesmo arquivo serve de entrada para os geradores e de referência para os validadores, simplificando a reprodutibilidade.

3.5. Métricas de Validação

A avaliação combina treze métricas em cinco famílias. As nove métricas de **similaridade** (apresentadas na Seção 2.2) são calculadas par a par entre a saída prevista e a referência, sempre na escala normalizada [0, 1]: similaridade

lexical (FuzzyWuzzy e TF-IDF), semântica (SBERT-PT `tgsc/sentence-transformer-ult5-pt-small`, LegalBERTimbau e `paraphrase-multilingual-mpnet-base-v2`), de distância (WMD sobre *embeddings* do FastText e do NILC) e voltadas para geração (BERTScore e ROUGE-L). A elas somam-se quatro métricas de **classificação** (acurácia, precisão, revocação e *F1-score*), que tratam a avaliação como extração de informação e tornam explícitas omissões e alucinações. Um elemento é *positivo* quando preenchido e *negativo* quando vazio (""); o critério de “casamento” é a similaridade semântica acima de um limiar de 0,7 em N1 e N2, e a igualdade exata (após normalização de caixa, acentos e pontuação) em N3. O limiar de 0,7 equilibra rigor e tolerância a paráfrases legítimas; como é aplicado igualmente a todos os modelos e experimentos, preserva a comparabilidade relativa entre eles.

3.6. Ambiente Computacional e Implementação

Todas as execuções foram conduzidas em uma única estação de trabalho (CPU Intel Core i9-13900K, 32 GB de RAM, GPU NVIDIA RTX 4080, Ubuntu 22.04, Python 3.11), com os modelos servidos pelo Ollama (`ollama serve`) e as métricas calculadas com `sentence-transformers`, `gensim`, `fuzzywuzzy` e `scikit-learn`. O *pipeline* foi implementado como um projeto Python modular, orquestrado por um *script* principal que expõe três caminhos (geração de dados, validação e geração de tabelas). O código organiza-se em pacotes por responsabilidade: `config/` (registro dos modelos e conexão com o Ollama), `prompts/` (os *prompts* em arquivos de texto, um por nível e um por operador no N3), `generates/` e `validates/` (um módulo por etapa, com estrutura simétrica), `utils/` (rotinas auxiliares de limpeza, emparelhamento e cálculo das métricas) e o `dataset.json` anotado. Essa separação mantém cada nível isolado e torna o *pipeline* reproduzível: trocar de modelo significa apenas alterar uma entrada de configuração, e cada experimento pode ser regenerado ou revalidado de forma independente. O código-fonte completo está disponível em <https://github.com/EikeSousa/Rase-PE>.

4. Resultados e Discussão

Esta seção apresenta os resultados da aplicação dos seis LLMs *open-source* sobre as 79 normas brasileiras do *dataset*, nos cinco experimentos: EN1 (segmentação N1 isolada), EN2 (identificação RASE com N1 de referência), EN1N2 (*pipeline* encadeado N1→N2), EN3 (estruturação JSON em isolamento, com N2 de referência) e EN1N2N3 (*pipeline* encadeado completo N1→N2→N3). Todos os números vêm diretamente dos arquivos *metrics/validate_*.json*, gerados pela rotina de validação descrita na Seção 3.5. Para cada experimento, o texto traz as médias agregadas por métrica, os resultados detalhados por modelo, os tempos de execução e uma discussão integrada.

Convenção de leitura das tabelas. Em todas as tabelas numéricas desta seção, o realce é feito **por coluna**: o melhor valor de cada coluna aparece em **verde** e o pior em **vermelho**. Nas tabelas de qualidade (similaridade e classificação), os valores em **negrito** são aqueles dentro do limiar de similaridade adotado na validação, isto é, $\geq 0,7$ (Seção 3.5). Já nas tabelas de tempo de execução, em que *menor é melhor*, o **verde** marca o modelo mais rápido e o **vermelho** o mais lento, e o negrito não se aplica.

4.1. Visão Geral dos Experimentos

A Tabela 2 resume as médias por métrica nos cinco experimentos, agregadas sobre os seis modelos. Três padrões já saltam aos olhos: as métricas baseadas em *embeddings* (SBERT, BERTimbau, Multilingual e BERTScore) mantêm desempenho consistentemente mais alto; as lexicais (FuzzyWuzzy, TF-IDF e ROUGE-L) perdem terreno à medida que se sai do EN1 e se entra no EN1N2; e o EN3 (N3 isolado) sobe a patamares próximos do EN1 graças ao vocabulário restrito dos campos JSON.

As métricas semânticas (SBERT, BERTimbau e Multilingual), junto do BERTScore, preservaram melhor o valor agregado ao longo dos cinco experimentos, por serem mais robustas a variações de superfície. As lexicais e o ROUGE-L sofrem com a reformulação produzida pelos LLMs. No N3, as métricas de similaridade textual sobem para patamares bem mais altos do que em N2 (saída quase-estruturada e curta), mas as métricas de classificação por correspondência exata, exigida em EN3, expõem dificuldades sistemáticas: a precisão macro fica abaixo de 4%. O EN1N2N3 confirma a tendência observada em EN1N2: as métricas semânticas absorvem bem a propagação de erros, enquanto as lexicais (TF-IDF e ROUGE-L) acusam queda visível.

4.2. Resultados por Experimento

4.2.1. EN1: Segmentação N1

No EN1, a comparação foi entre as regras N1 geradas pelos LLMs e o N1 de referência. As métricas semânticas ficaram bem acima das lexicais (Multilingual 0,850, BERTimbau 0,816 e SBERT 0,801, contra TF-IDF 0,636 e

Tabela 2

Médias por métrica nos cinco experimentos, agregadas sobre os seis modelos.

Métrica	EN1	EN2	EN1N2	EN3	EN1N2N3
<i>Similaridade lexical</i>					
FuzzyWuzzy	0,475	0,461	0,464	0,847	0,617
TF-IDF	0,636	0,312	0,417	0,597	0,514
<i>Similaridade semântica</i>					
SBERT	0,801	0,490	0,594	0,906	0,777
BERTimbau	0,816	0,542	0,631	0,836	0,752
Multilingual	0,850	0,595	0,679	0,873	0,793
<i>Distância semântica (WMD)</i>					
WMD_ft	0,716	0,580	0,625	0,846	0,757
WMD_nilc	0,688	0,546	0,595	0,842	0,742
<i>Métricas voltadas para geração</i>					
BERTScore	0,866	0,753	0,790	0,880	0,843
ROUGE-L	0,599	0,308	0,404	0,765	0,616
<i>Métricas de classificação</i>					
Acurácia	0,475	0,273	—	0,576	—
Precisão	0,545	0,202	—	0,024	—
Revocação	0,774	0,198	—	0,380	—
F1-score	0,631	0,197	—	0,043	—

Tabela 3

Resultados por modelo em EN1 (similaridade e classificação).

Modelo	Lexical		Semântica			WMD		Geração	
	FW	TF-IDF	SBERT	BERTimbau	Multi.	WMD_ft	WMD_nilc	BERTScore	ROUGE-L
Alpaca	0,473	0,561	0,748	0,762	0,803	0,694	0,664	0,841	0,540
Dolphin	0,641	0,756	0,873	0,891	0,913	0,774	0,751	0,905	0,703
Gemma	0,412	0,695	0,842	0,850	0,885	0,726	0,699	0,883	0,648
Llama	0,422	0,672	0,835	0,844	0,873	0,711	0,684	0,871	0,605
Mistral	0,534	0,704	0,842	0,874	0,899	0,726	0,699	0,888	0,651
Qwen	0,367	0,429	0,668	0,673	0,726	0,663	0,632	0,808	0,448
<i>Métricas de classificação</i>									
Modelo	Acurácia		Precisão		Revocação		F1		
Alpaca	0,419		0,512		0,699		0,591		
Dolphin	0,676		0,812		0,801		0,806		
Gemma	0,466		0,486		0,917		0,636		
Llama	0,446		0,479		0,865		0,616		
Mistral	0,610		0,670		0,872		0,758		
Qwen	0,235		0,311		0,487		0,380		

FuzzyWuzzy 0,475), o que indica que a segmentação tolera reformulação lexical quando o sentido é preservado. O **Dolphin** dominou: liderou em *todas* as nove métricas de similaridade e em três das quatro de classificação, com o melhor desempenho isolado do experimento. Os números detalhados estão na Tabela 3.

A Figura 2 apresenta o comparativo visual entre os seis modelos nas nove métricas de similaridade em EN1.

4.2.2. EN2: Identificação dos Operadores RASE

O EN2 mediu a identificação e a estruturação dos operadores RASE a partir do N1 de referência. As médias caíram em relação ao EN1 (SBERT 0,490; Multilingual 0,595), o que reflete a maior dificuldade da formalização estruturada. As métricas de classificação caem ainda mais (F1 macro 0,197): os modelos têm dificuldade real em distinguir Seleção e Exceção. O **Llama** lidera nas métricas semânticas e lexicais; o **Dolphin** fica com a melhor F1 macro (0,286). Os números detalhados estão na Tabela 4.

A Figura 3 consolida o desempenho dos seis modelos nas nove métricas de similaridade em EN2.

4.2.3. EN1N2: Pipeline Encadeado

O EN1N2 testou a robustez do *pipeline* totalmente automatizado: o N1 produzido pelo modelo na primeira etapa alimenta o N2 do mesmo modelo. Os resultados confirmam a hipótese de **propagação de erros**, mas com uma surpresa. As médias caem em relação ao EN2 em parte das métricas e se mantêm próximas ou até melhoram em outras, caso

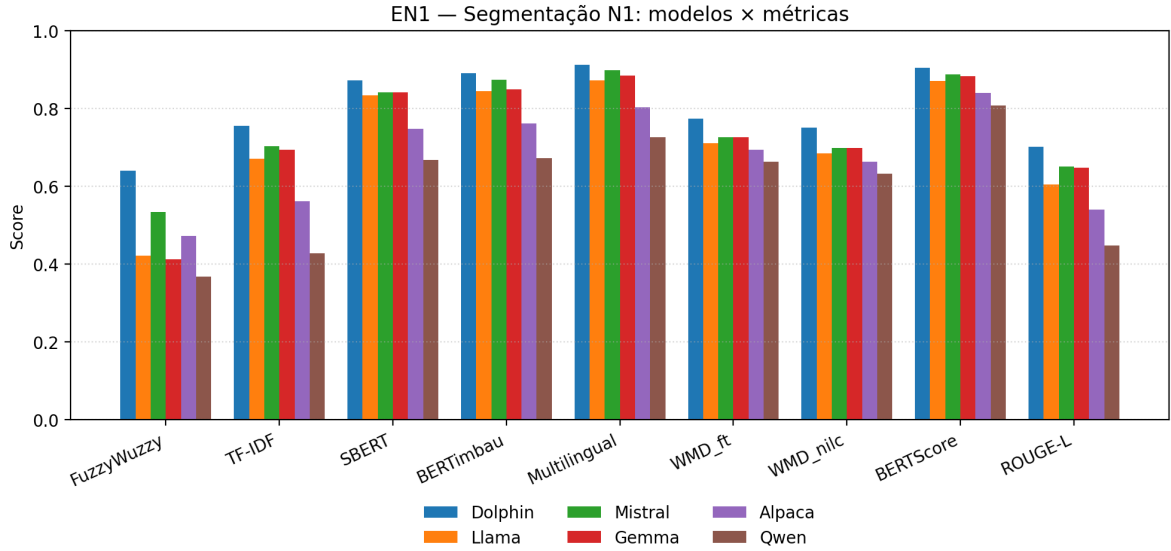


Figura 2: Comparativo modelo × métrica em EN1.

Tabela 4

Resultados por modelo em EN2 (similaridade e classificação macro por operador).

Modelo	Lexical			Semântica		WMD		Geração	
	FW	TF-IDF	SBERT	BERTimbau	Multi.	WMD_ft	WMD_nilc	BERTScore	ROUGE-L
Alpaca	0,334	0,168	0,386	0,418	0,484	0,522	0,486	0,702	0,172
Dolphin	0,511	0,423	0,557	0,621	0,665	0,620	0,589	0,786	0,412
Gemma	0,384	0,364	0,520	0,579	0,625	0,608	0,577	0,772	0,350
Llama	0,638	0,447	0,595	0,657	0,685	0,617	0,586	0,796	0,427
Mistral	0,563	0,385	0,575	0,607	0,665	0,607	0,575	0,779	0,377
Qwen	0,336	0,083	0,304	0,369	0,444	0,496	0,462	0,684	0,111
Métricas de classificação (macro por operador)									
Modelo	Acurácia		Precisão		Revocação		F1		
Alpaca	0,081		0,103		0,109		0,105		
Dolphin	0,339		0,301		0,276		0,286		
Gemma	0,182		0,252		0,300		0,265		
Llama	0,439		0,237		0,250		0,243		
Mistral	0,434		0,268		0,214		0,238		
Qwen	0,160		0,051		0,039		0,044		

Tabela 5

Resultados por modelo em EN1N2 (similaridade).

Modelo	Lexical			Semântica		WMD		Geração	
	FW	TF-IDF	SBERT	BERTimbau	Multi.	WMD_ft	WMD_nilc	BERTScore	ROUGE-L
Alpaca	0,375	0,285	0,494	0,523	0,581	0,573	0,538	0,742	0,282
Dolphin	0,549	0,529	0,662	0,711	0,749	0,673	0,644	0,826	0,509
Gemma	0,391	0,458	0,615	0,656	0,701	0,646	0,617	0,804	0,436
Llama	0,570	0,521	0,677	0,721	0,749	0,649	0,620	0,821	0,488
Mistral	0,559	0,489	0,667	0,698	0,740	0,648	0,618	0,817	0,468
Qwen	0,342	0,219	0,448	0,478	0,551	0,562	0,530	0,731	0,240

das semânticas: Multilingual sobe de 0,595 para 0,679; BERTimbau de 0,542 para 0,631; SBERT de 0,490 para 0,594. O motivo é que o *pipeline* produz saídas mais próximas das frases originais e, por isso, mais alinhadas ao texto de referência no nível semântico, ainda que perca precisão na atribuição dos operadores. A liderança é compartilhada: o **Dolphin** domina TF-IDF, WMD_ft, WMD_nilc, BERTScore e ROUGE-L; o **Llama** lidera SBERT e empata em Multilingual. Os números completos estão na Tabela 5.

A Figura 4 apresenta o comparativo visual em EN1N2, evidenciando a alternância de liderança entre Dolphin e Llama por métrica.

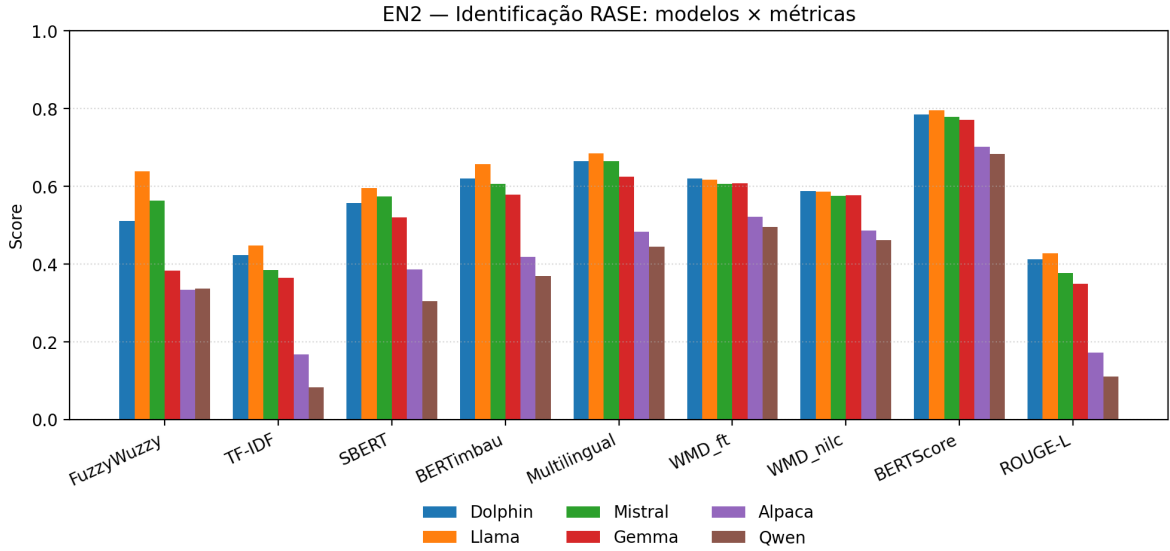


Figura 3: Comparativo modelo × métrica em EN2.

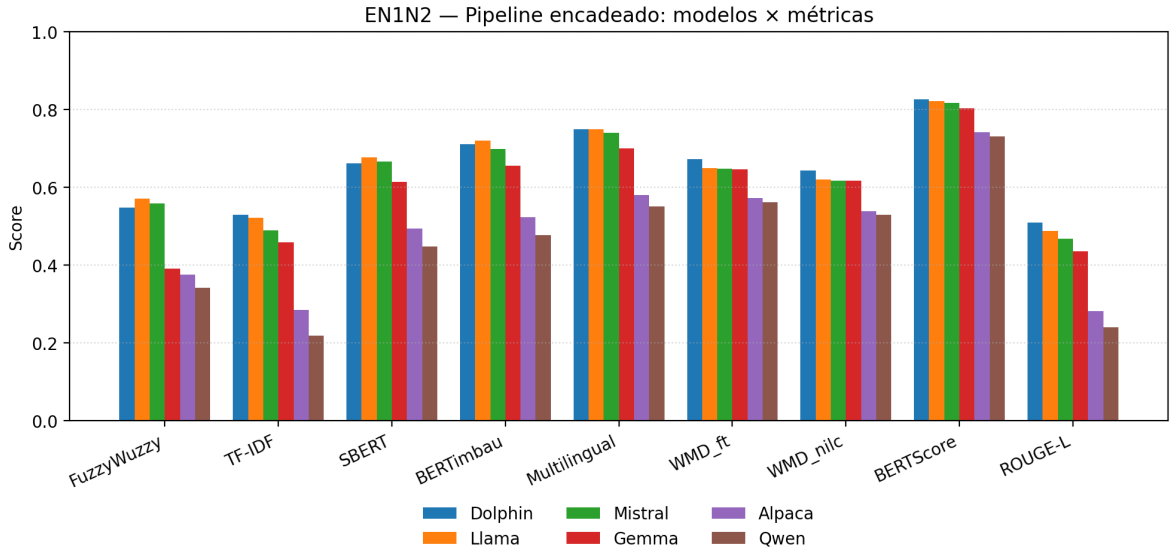


Figura 4: Comparativo modelo × métrica em EN1N2.

4.2.4. EN3: Estruturação Semântica em JSON com N2 de referência

O nível N3 (estruturação semântica em JSON) foi avaliado em dois cenários complementares: o EN3, em isolamento, recebendo como entrada os operadores N2 de referência; e o EN1N2N3, no *pipeline* encadeado de ponta a ponta, em que o N1 gerado alimenta o N2 do mesmo modelo, que por sua vez alimenta o N3 do mesmo modelo. Em ambos os cenários, a comparação não é entre *strings* normativas inteiras, e sim campo a campo (*type*, *object*, *property*, *comparison*, *target* e *unit*), sobre a serialização canônica do JSON gerado contra o JSON anotado.

Como a saída é *quase-estruturada* e composta por campos curtos com vocabulário restrito, as métricas de similaridade textual sobem para patamares bem mais altos do que em N1 e N2 (ver Tabela 6). As métricas de classificação por correspondência exata, em compensação, mostram um quadro bem mais duro: a precisão macro fica abaixo de 4%. Isso sinaliza que os modelos preservam o significado, mas variam o vocabulário em relação à referência,

Tabela 6

Resultados por modelo em EN3 (similaridade e classificação macro por campo).

Modelo	Lexical		Semântica			WMD		Geração	
	FW	TF-IDF	SBERT	BERTimbau	Multi.	WMD ft	WMD nilc	BERTScore	ROUGE-L
Alpaca	0,729	0,395	0,846	0,745	0,798	0,756	0,751	0,815	0,624
Dolphin	0,860	0,694	0,912	0,848	0,887	0,842	0,839	0,901	0,799
Gemma	0,751	0,480	0,858	0,755	0,823	0,762	0,757	0,810	0,652
Llama	0,892	0,736	0,947	0,897	0,928	0,915	0,912	0,925	0,871
Mistral	0,877	0,689	0,938	0,891	0,914	0,903	0,900	0,922	0,842
Qwen	0,872	0,590	0,937	0,877	0,889	0,898	0,895	0,905	0,804

Métricas de classificação (macro por campo)

Modelo	Acurácia	Precisão	Revocação	F1
Alpaca	0,283	0,012	0,434	0,024
Dolphin	0,705	0,034	0,447	0,062
Gemma	0,292	0,017	0,603	0,033
Llama	0,725	0,038	0,361	0,067
Mistral	0,760	0,037	0,371	0,066
Qwen	0,692	0,004	0,063	0,008

Tabela 7

Resultados por modelo em EN1N2N3 (similaridade sobre o JSON estruturado).

Modelo	Lexical		Semântica			WMD		Geração	
	FW	TF-IDF	SBERT	BERTimbau	Multi.	WMD ft	WMD nilc	BERTScore	ROUGE-L
Alpaca	0,525	0,345	0,687	0,645	0,701	0,676	0,658	0,782	0,471
Dolphin	0,685	0,600	0,811	0,794	0,832	0,776	0,762	0,871	0,683
Gemma	0,536	0,419	0,747	0,710	0,767	0,712	0,697	0,807	0,553
Llama	0,701	0,630	0,834	0,823	0,853	0,804	0,790	0,882	0,711
Mistral	0,693	0,600	0,830	0,814	0,845	0,801	0,788	0,879	0,691
Qwen	0,561	0,490	0,750	0,724	0,762	0,773	0,759	0,838	0,590

por exemplo `target = "público"` em vez de `target = "uso público"`. O **Llama** dominou: liderou em *todas* as nove métricas de similaridade, com SBERT 0,947 e Multilingual 0,928. Mistral, Dolphin e Qwen ficam próximos em métricas semânticas, mostrando que a etapa N3 isolada é a mais homogênea entre os modelos avaliados.

O setup foi idêntico ao dos experimentos anteriores: os mesmos seis LLMs servidos via Ollama, os mesmos *prompts few-shot* por operador, o mesmo *dataset* de 79 normas. Saídas por modelo ficam em arquivos JSON dedicados (`generate_n3_<modelo>.json`); o agregado vive em um único arquivo de métricas (`validate_n3.json`). A correspondência exata por campo é mais dura que o limiar de similaridade adotado em N1 e N2, o que derruba a precisão observada, em especial nos modelos com menor aderência ao esquema (Alpaca e Qwen).

4.2.5. EN1N2N3: Pipeline Encadeado Completo

O EN1N2N3 é o *pipeline* automatizado de ponta a ponta. Este é o cenário que replica o uso real em produção: a única entrada é o texto bruto da norma, e a saída final é o JSON estruturado por operador. Como cada etapa acumula transformações, é também o experimento mais sujeito à **propagação de erros**: as médias caem em relação ao EN3 isolado em todas as métricas (SBERT 0,777 vs. 0,906; Multilingual 0,793 vs. 0,873; BERTScore 0,843 vs. 0,880). Ainda assim, os patamares absolutos permanecem superiores aos observados em EN2 e EN1N2, beneficiados pelo vocabulário restrito do JSON. A Tabela 7 traz os resultados por modelo.

A saída de cada modelo é persistida em um arquivo JSON próprio (`generate_n1n2n3_<modelo>.json`), e as métricas agregadas ficam em um arquivo único (`validate_n1n2n3.json`). O **Llama** mantém a liderança que conquistou em EN3 isolado, agora em todas as nove métricas de similaridade; **Mistral** fica imediatamente atrás, com diferenças marginais (Multilingual 0,845 vs. 0,853). Em linha com o padrão observado em EN1N2, Llama, Mistral e Dolphin preservam vantagem relativa também na cadeia completa, ao passo que Alpaca e Gemma acumulam queda em todas as métricas e entregam o pior desempenho global, evidenciando que falhas em N1/N2 contaminam a estruturação final em N3 mesmo nas métricas semânticas.

4.3. Resultados por Modelo

A Tabela 8 traz a média global por modelo, agregando as nove métricas de similaridade textual sobre os cinco experimentos (EN1, EN2, EN1N2, EN3 e EN1N2N3). O ranking final foge do esperado: o líder é o **Llama**, sustentado pela liderança absoluta em EN3 e EN1N2N3 (ambas as etapas N3) e pelo melhor desempenho em EN2. O **Dolphin**

Tabela 8

Média global por modelo (nove métricas de similaridade × cinco experimentos).

Modelo	EN1	EN2	EN1N2	EN3	EN1N2N3	Média Global
Llama	0,724	0,605	0,646	0,892	0,781	0,730
Dolphin	0,801	0,576	0,650	0,843	0,757	0,725
Mistral	0,757	0,570	0,634	0,875	0,771	0,721
Gemma	0,738	0,531	0,591	0,739	0,661	0,652
Qwen	0,602	0,366	0,456	0,852	0,694	0,594
Alpaca	0,676	0,408	0,488	0,718	0,610	0,580

fica em segundo: domina o EN1 com folga e mantém desempenho competitivo nas etapas seguintes. Mistral aparece em terceiro, seguido por Gemma, Qwen e Alpaca.

A seguir, destacam-se as observações por modelo.

Llama. Lidera a média global (0,730) e domina os experimentos do nível N3: em EN3 isolado ficou à frente em *todas* as nove métricas (SBERT 0,947; Multilingual 0,928; BERTScore 0,925); em EN1N2N3, repetiu a liderança nas nove métricas (SBERT 0,834; Multilingual 0,853; BERTScore 0,882). Foi também o melhor em EN2 em sete métricas: FuzzyWuzzy (0,638), TF-IDF (0,447), SBERT (0,595), BERTimbau (0,657), Multilingual (0,685), BERTScore (0,796) e ROUGE-L (0,427), além da maior acurácia macro de classificação no mesmo experimento (0,439). No EN1N2, liderou em FuzzyWuzzy (0,570), SBERT (0,677) e BERTimbau (0,721), empatando com o Dolphin em Multilingual (0,749). É a escolha quando se tolera o custo computacional.

Dolphin. Foi o melhor segmentador em EN1, com domínio absoluto nas nove métricas (BERTimbau 0,891; Multilingual 0,913; SBERT 0,873; BERTScore 0,905) e a melhor F1 macro de classificação no experimento (0,806). No EN2, liderou em WMD_ft (0,620), WMD_nilc (0,589) e F1 macro (0,286). No EN1N2, ficou à frente em TF-IDF (0,529), WMD_ft (0,673), WMD_nilc (0,644), BERTScore (0,826) e ROUGE-L (0,509). Em EN3 e EN1N2N3, mantém desempenho competitivo (SBERT 0,912 e 0,811, respectivamente), embora atrás de Llama e Mistral. Soma-se a isso o baixo custo computacional (Seção 4.5), o que rende a melhor relação custo-benefício observada.

Mistral. Desempenho consistente em todos os cenários, e o mais regular da bateria: terceiro colocado na média global (0,721), a menos de um ponto percentual do líder. Ficou próximo dos líderes em EN1 (Multilingual 0,899; BERTimbau 0,874) e em EN1N2 (Multilingual 0,740; BERTimbau 0,698), entregou a segunda melhor acurácia macro do EN2 (0,434), e fica colado ao Llama em EN3 (SBERT 0,938) e em EN1N2N3 (SBERT 0,830).

Gemma. Aparece como líder em revocação macro de classificação tanto no EN1 (0,917) quanto no EN2 (0,300), ou seja, boa cobertura, com precisão menor. As médias semânticas ficam em patamar intermediário.

Alpaca. Desempenho fraco em todos os experimentos, com queda acentuada no EN2 (SBERT 0,386; F1 macro 0,105).

Qwen. Pior desempenho geral. Colapsou no EN2 (TF-IDF 0,083; F1 macro 0,044) e teve a maior latência do *pipeline* encadeado (ver Seção 4.5). Pesa também a tendência do modelo a gerar saídas em chinês simplificado, fora do idioma esperado.

4.4. Análise por Métrica

Métricas lexicais. FuzzyWuzzy e TF-IDF caem claramente de EN1 para EN2 (FuzzyWuzzy: 0,475 → 0,461; TF-IDF: 0,636 → 0,312). Como dependem de sobreposição lexical, essas métricas punem as reformulações sintáticas e as sinonímias geradas pelos LLMs. Em EN3 e EN1N2N3, voltam a subir (FuzzyWuzzy: 0,847 e 0,617; TF-IDF: 0,597 e 0,514), beneficiadas pelo vocabulário curto e restrito do JSON.

Métricas semânticas. SBERT, BERTimbau e Multilingual mantêm médias altas e estáveis. O BERTimbau, treinado sobre texto jurídico em português, fica ligeiramente acima do SBERT em quase todos os modelos e experimentos (EN1: 0,816 vs. 0,801; EN1N2: 0,631 vs. 0,594), evidência prática do ganho de usar *embeddings* de domínio para validar transformações normativas. Em EN3, atingem os valores mais altos da bateria (SBERT 0,906; Multilingual 0,873), porque os campos JSON contêm sintagmas curtos com baixa variação léxica.

Métricas voltadas para geração. O BERTScore teve a maior média absoluta em EN2 (0,753) e em EN1N2 (0,790), graças ao alinhamento ótimo entre *tokens* mesmo quando há forte reformulação. Em EN3 e EN1N2N3, segue alto (0,880 e 0,843). Já o ROUGE-L comportou-se como métrica lexical, com queda acentuada em EN2 (0,308) e recuperação em EN3 (0,765).

Tabela 9

Tempo total de geração por modelo em N1.

Modelo	Tempo (s)	Tempo (h:mm:ss)
Alpaca	104,55	0:01:44
Dolphin	62,65	0:01:02
Gemma	133,04	0:02:13
Llama	5 676,97	1:34:36
Mistral	70,04	0:01:10
Qwen	309,59	0:05:09

Tabela 10

Tempo total de geração por modelo em N2.

Modelo	Tempo (s)	Tempo (h:mm:ss)
Alpaca	249,10	0:04:09
Dolphin	124,69	0:02:04
Gemma	244,19	0:04:04
Llama	6 895,71	1:54:55
Mistral	123,63	0:02:03
Qwen	6 515,09	1:48:35

Tabela 11

Tempo total de geração por modelo em N1N2.

Modelo	Tempo (s)	Tempo (h:mm:ss)
Alpaca	330,84	0:05:30
Dolphin	104,28	0:01:44
Gemma	427,22	0:07:07
Llama	9 450,14	2:37:30
Mistral	138,39	0:02:18
Qwen	10 093,84	2:48:13

Métricas de distância (WMD). Desempenho intermediário, entre lexicais e semânticas. Os valores foram mais altos em EN1 (WMD_ft 0,716; WMD_nilc 0,688) e em EN3 (WMD_ft 0,846; WMD_nilc 0,842), refletindo a proximidade lexical maior entre as saídas e o texto/JSON de referência nesses dois cenários.

Métricas de classificação. Em EN1, a F1 macro chega a 0,806 (Dolphin), o que indica que a maior parte das sentenças atômicas é segmentada corretamente. Em EN2, a F1 desaba para 0,286 no melhor caso (Dolphin), com colapso em Seleção e Exceção: F1 próxima de zero para todos os modelos. Esses dois operadores são especialmente difíceis de identificar mesmo para os líderes. Em EN3, a baixa precisão por correspondência exata (precisão macro abaixo de 4% para todos os modelos) mostra que o emparelhamento *token a token* é severo demais para um campo cuja semântica resiste a variações léxicas legítimas.

4.5. Tempo de Execução

O tempo total de geração foi medido por modelo e por experimento. A Tabela 9 traz os tempos em N1; a Tabela 10, em N2; a Tabela 11, no *pipeline* encadeado N1N2; a Tabela 12 traz os tempos do EN3 (N3 isolado, com N2 de referência como entrada); e a Tabela 13, do *pipeline* completo EN1N2N3.

Os tempos expõem um *trade-off* nítido entre qualidade e custo computacional. Em EN1 e EN1N2, Llama e Qwen são os mais lentos: respectivamente 1h34min e 5min em N1; 2h37min e 2h48min em N1N2. Em EN3 e EN1N2N3, o quadro se inverte: o **Gemma** passa a ser o mais oneroso (1h39min em N3; 1h46min em N1N2N3), enquanto Llama, Mistral e Qwen ficam em torno de 10 a 14 minutos. O Dolphin combina alta qualidade com tempos na casa de 1 a 13 minutos em todos os experimentos (62 s em N1; 1m44s em N1N2; 9m45s em N3; 12m38s em N1N2N3), e segue como a melhor relação custo-benefício observada na bateria.

Tabela 12

Tempo total de geração por modelo em N3 (EN3 isolado).

Modelo	Tempo (s)	Tempo (h:mm:ss)
Alpaca	2 104,92	0:35:05
Dolphin	584,96	0:09:45
Gemma	5 982,38	1:39:42
Llama	800,78	0:13:21
Mistral	627,68	0:10:28
Qwen	847,41	0:14:07

Tabela 13Tempo total de geração por modelo em N1N2N3 (*pipeline* completo).

Modelo	Tempo (s)	Tempo (h:mm:ss)
Alpaca	1 395,35	0:23:15
Dolphin	757,65	0:12:38
Gemma	6 364,48	1:46:04
Llama	808,62	0:13:29
Mistral	637,88	0:10:38
Qwen	808,72	0:13:29

4.6. Discussão Integrada

A leitura conjunta dos três experimentos e das treze métricas permite consolidar cinco observações.

1. *Embeddings* semânticos e BERTScore são as métricas mais adequadas a esta tarefa. SBERT, BERTimbau, Multilingual e BERTScore capturam equivalência mesmo quando há reformulação lexical pesada, e são as métricas mais estáveis ao longo dos experimentos. O BERTimbau, treinado em corpus jurídico em PT-BR, fica ligeiramente acima do SBERT, evidência do ganho de *embeddings* de domínio.

2. O *pipeline* encadeado não degrada todas as métricas de forma uniforme. A queda esperada pela propagação de erros aparece nas lexicais (FuzzyWuzzy, TF-IDF e ROUGE-L). Já nas semânticas, o EN1N2 chega a *superar* o EN2 isolado nas médias agregadas (Multilingual 0,679 vs. 0,595): o N1 produzido pelo próprio modelo tende a gerar frases próximas das originais, o que favorece o emparelhamento semântico, mesmo quando o N2 erra na atribuição dos operadores. As métricas de classificação confirmam, porém, que esse ganho não se converte em classificação RASE melhor: o *pipeline* continua dependendo de saída atomizada e corretamente rotulada para que a verificação automática faça sentido.

3. Há perfis complementares entre os modelos líderes. O Llama lidera a qualidade global (0,730), com domínio absoluto sobre o nível N3 (as nove métricas em EN3 e em EN1N2N3) e também sobre o EN2; é a escolha quando se tolera o custo computacional. O Dolphin (0,725) entrega o melhor custo-benefício: domina o EN1, fica à frente nas métricas de distância do EN1N2 e tem o menor tempo de geração entre os líderes. O Mistral (0,721) é o terceiro e o mais consistente: nunca lidera com folga, mas nunca está longe.

4. Qwen e Alpaca não são recomendados. Ambos têm desempenho consistentemente inferior em todas as métricas e experimentos. O Qwen ainda registra a maior latência do *pipeline* encadeado (2h48min) e tende a gerar saídas fora do idioma esperado.

5. A decomposição em três níveis foi decisiva. Separar segmentação (N1), classificação RASE (N2) e estruturação JSON (N3) viabilizou saídas válidas sem utilizar *Fine-Tuning* nem *Retrieval-Augmented Generation*, ou seja, Engenharia de Prompt sozinha basta para alcançar similaridade semântica perto de 0,90 nos melhores cenários (Dolphin em EN1, com BERTimbau 0,891; Llama em N3 isolado, com SBERT 0,947). O que sobra de dificuldade está concentrado na classificação correta dos operadores Seleção e Exceção em N2, as classes mais difíceis para todos os modelos avaliados.

5. Conclusão

Este trabalho investigou a aplicação de seis LLMs *open-source* (Llama, Dolphin, Gemma, Mistral, Alpaca e Qwen) na conversão automática de normas técnicas brasileiras da AEC para o formato RASE, usando **apenas Engenharia**

de Prompt. A tarefa foi decomposta em três níveis sucessivos de normalização (N1, N2 e N3) e avaliada em cinco experimentos (EN1, EN2, EN1N2, EN3 e EN1N2N3), sobre um *dataset* de 79 normas anotadas. A avaliação combinou treze métricas: nove de similaridade textual (lexicais, semânticas, de distância e voltadas para geração) e quatro de classificação, derivadas da matriz de confusão.

5.1. Síntese dos Resultados

A decomposição em três níveis viabilizou a conversão de normas AEC em representações RASE sem recorrer a *Fine-Tuning* nem a *Retrieval-Augmented Generation*. Engenharia de Prompt sozinha bastou para alcançar similaridade semântica perto de 0,90 nos melhores cenários (Dolphin em EN1, com BERTimbau 0,891 e Multilingual 0,913; Llama em EN3 isolado, com SBERT 0,947; e novamente Llama em EN1N2N3, com SBERT 0,834 mesmo na cadeia completa). As métricas baseadas em *embeddings* (SBERT, BERTimbau, Multilingual) e o BERTScore foram as mais adequadas para validar transformações normativas, porque toleram reformulações que preservam o sentido. O BERTimbau manteve ganho consistente sobre o SBERT, evidência prática do valor de *embeddings* de domínio jurídico em PT-BR.

No agregado das nove métricas de similaridade sobre os cinco experimentos, o **Llama** fechou com a melhor média global (0,730), puxado pela liderança absoluta em EN3 e EN1N2N3 (as duas etapas do nível N3) e em EN2. O **Dolphin** ficou em segundo (0,725), com a melhor relação custo-benefício: cerca de noventa vezes mais rápido que o Llama no *pipeline* encadeado, e líder absoluto em EN1. O Mistral fechou em terceiro (0,721), o mais consistente da bateria. Qwen e Alpaca apresentaram desempenho insatisfatório nas etapas mais sensíveis a falhas semânticas (EN2 e EN1N2), mas se recuperaram em EN3, beneficiados pelo vocabulário restrito do JSON. As métricas de classificação confirmaram um achado importante: os operadores Seleção e Exceção são as classes mais difíceis em N2 para todos os modelos, com F1 próxima de zero; já em EN3, a correspondência exata por campo derruba a precisão macro para abaixo de 4%, apontando a necessidade de critérios de classificação mais flexíveis para o nível N3.

5.2. Contribuições

A pesquisa entrega cinco contribuições centrais:

- A definição operacional de três níveis de normalização sobre a metodologia RASE: **N1** (segmentação atômica), **N2** (identificação dos operadores RASE) e **N3** (estruturação semântica em JSON), com *prompts* dedicados a cada nível e a cada operador.
- Uma comparação sistemática de seis LLMs *open-source* servidos localmente via Ollama, na tarefa de conversão de normas brasileiras da AEC, sob protocolo experimental idêntico (mesmo *dataset*, mesmos *prompts*, mesmo *hardware*).
- Um conjunto reproduzível de *prompts* especializados por nível e por operador, organizado em arquivos de texto dedicados, pronto para ser reutilizado e estendido em pesquisas futuras.
- Um *dataset* anotado de 79 normas brasileiras, com referência em todos os níveis (N1, N2 e N3) e por operador RASE, viabilizando avaliações replicáveis.
- Uma avaliação multimétrica com treze métricas (nove de similaridade textual e quatro de classificação), entregando um retrato multidimensional do desempenho de cada modelo, com detalhamento por instância em *metrics/details/* para apoiar reanálises.

5.3. Limitações

A pesquisa tem limitações que delimitam o alcance das conclusões:

- O *dataset* é pequeno (79 normas) e está concentrado em normas brasileiras de acessibilidade (NBR 9050 e similares), de modo que os resultados podem mudar em outros domínios normativos.
- As métricas de classificação dependem do critério de acerto adotado: limiar sobre similaridade semântica em N1 e N2 (parcialmente arbitrário) e correspondência exata em N3 (rígida demais para campos com variações léxicas legítimas). Critérios mais flexíveis em N3 e múltiplos limiares em N1 e N2 ficam como linha de aprofundamento.

- A classificação de Seleção e Exceção em N2 obteve F1 próxima de zero para todos os modelos, sinalizando uma limitação estrutural da formulação atual do *prompt* N2: nenhum dos seis LLMs conseguiu superá-la.
- Não houve aplicação prática em *software* BIM. A validação fim a fim em projetos reais ainda está em aberto; o ciclo completo de verificação automática de conformidade precisa ser fechado.
- Todos os experimentos rodaram em uma única estação de trabalho, sem análise de variância entre execuções, sem variação de temperatura e sem variação de *seed*.

5.4. Trabalhos Futuros

Algumas direções de pesquisa futura consideradas promissoras:

- **Aplicação em BIM:** integrar os resultados ao Revit ou ao Dynamo e fechar o ciclo de verificação automática de conformidade em projetos reais.
- **Prompt N2 dedicado a Seleção e Exceção:** redesenhar o *prompt* N2 ou introduzir uma etapa específica para esses dois operadores. Ambos colapsaram em F1 próxima de zero nos seis modelos, o que torna esse o ponto de maior alavancagem.
- **Fine-Tuning e RAG:** explorar essas técnicas como evoluções naturais da abordagem baseada apenas em *prompts*, medindo o ganho em qualidade e em custo computacional, em especial para os operadores difíceis citados acima.
- **Ampliação do dataset:** incluir um espectro maior de NBRs (estruturais, hidráulicas, elétricas) e expandir para normas internacionais.
- **Avaliação humana:** complementar as métricas automáticas com avaliação por especialistas da AEC, capazes de capturar nuances semânticas que escapam aos *embeddings*.
- **Estudo de variância:** rodar múltiplas execuções por modelo, variando temperatura e *seed*, para quantificar a estabilidade das saídas.
- **Critérios alternativos de classificação em N3:** trocar a correspondência exata por correspondência normalizada (*lowercase*, remoção de *stopwords*, comparação por sinônimos) e revisitar Acurácia, Precisão, Revocação e F1 em N3.
- **Otimização do pipeline:** estudar estratégias de mitigação da propagação de erros em EN1N2 e em EN1N2N3: reescrita iterativa, *self-consistency* ou roteamento de operadores difíceis para o modelo mais adequado (*ensemble* Dolphin/Llama por nível).

No conjunto, o trabalho mostra que LLMs *open-source*, somados a uma decomposição cuidadosa da tarefa e a *prompts* especializados, são alternativa viável e acessível para automatizar a conversão de normas técnicas da AEC em representações computáveis, abrindo caminho para a verificação automática de conformidade em projetos baseados em BIM.

CRedit authorship contribution statement

Eike Natan Sousa Brito: Conceituação, Metodologia, Software, Redação - rascunho original. **André Carvalho:** Supervisão, Redação - revisão e edição. **Marco Antônio Brasiel Sampaio:** Supervisão, Redação - revisão e edição.

Referências

- , 2018. ISO 19650-1: Organization and digitization of information about buildings and civil engineering works, including building information modelling (BIM) – Information management using building information modelling – Part 1: Concepts and principles. International Organization for Standardization. Geneva.
- , 2020. NBR 9050: Acessibilidade a edificações, mobiliário, espaços e equipamentos urbanos. Associação Brasileira de Normas Técnicas. Rio de Janeiro.
- AI@Meta, 2024. The Llama 3 herd of models. Model Card. URL: https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md.

- Alawadhi, M., Yan, W., 2023. Deep learning from parametrically generated virtual buildings for real-world object recognition. arXiv preprint arXiv:2302.05283 URL: <https://arxiv.org/abs/2302.05283>.
- Beach, T.H., Rezgui, Y., Li, H., Kasim, T., 2015. A rule-based semantic approach for automated regulatory compliance in the construction sector. Expert Systems with Applications 42, 5219–5231. doi:10.1016/j.eswa.2015.02.029.
- Bojanowski, P., Grave, E., Joulin, A., Mikolov, T., 2017. Enriching word vectors with subword information. Transactions of the Association for Computational Linguistics 5, 135–146.
- Brown, T.B., Mann, B., Ryder, N., Subbiah, M., et al., 2020. Language models are few-shot learners. URL: <https://arxiv.org/abs/2005.14165>, arXiv:2005.14165.
- Davenport, T.H., Patil, D.J., 2012. Data scientist: The sexiest job of the 21st century. Harvard Business Review 90, 70–76. URL: <https://hbr.org/2012/10/data-scientist-the-sexiest-job-of-the-21st-century>.
- Dimyadi, J., Amor, R., 2013. Automated building code compliance checking – where is it at?, in: Proceedings of the 19th International CIB World Building Congress, Brisbane. pp. 172–185.
- Eastman, C., Lee, J.M., Jeong, Y.S., Lee, J.K., 2009. Automatic rule-based checking of building designs. Automation in Construction 18, 1011–1033. doi:10.1016/j.autcon.2009.07.002.
- Eastman, C., Teicholz, P., Sacks, R., 2011. BIM Handbook: A Guide to Building Information Modeling for Owners, Managers, Designers, Engineers and Contractors. 2 ed., Wiley, New Jersey.
- Fuchs, S., Witbrock, M., Dimyadi, J., Amor, R., 2024. Using large language models for the interpretation of building regulations. arXiv preprint arXiv:2407.21060 URL: <https://arxiv.org/abs/2407.21060>.
- Hjelseth, E., 2015. BIM-based model checking (BMC), in: Building Information Modeling: Applications and Practices. ASCE, pp. 33–61. doi:10.1061/9780784413982.ch02.
- Hjelseth, E., Nisbet, N., 2011. Capturing normative constraints by use of the semantic mark-up RASE methodology, in: Proceedings of the CIB W78-W102 International Conference. URL: <https://api.semanticscholar.org/CorpusID:62759547>.
- Jurafsky, D., Martin, J.H., 2009. Speech and Language Processing. 2 ed., Prentice-Hall.
- Kusner, M.J., Sun, Y., Kolkin, N.I., Weinberger, K.Q., 2015. From word embeddings to document distances, in: Proceedings of the 32nd International Conference on Machine Learning (ICML), pp. 957–966.
- Lin, C.Y., 2004. ROUGE: A package for automatic evaluation of summaries, in: Text Summarization Branches Out: Proceedings of the ACL-04 Workshop, Association for Computational Linguistics. pp. 74–81.
- Lin, W.Y., 2023. Prototyping a chatbot for site managers using building information modeling (BIM) and natural language understanding (NLU) techniques. Sensors 23, 2942. doi:10.3390/s23062942.
- Liu, Y., Zhang, X., Li, H., 2022. BIM, machine learning and design rules to improve the assembly process. Sustainability 14, 288. URL: <https://www.mdpi.com/2071-1050/14/1/288>.
- Manning, C.D., Raghavan, P., Schütze, H., 2008. Introduction to Information Retrieval. Cambridge University Press, Cambridge, UK.
- Mendonça, E.A.d., Manzione, L., 2020. Conversão de regras de acessibilidade pela metodologia RASE para verificação automática em modelo BIM.
- Ollama, 2024. Ollama: Get up and running with large language models locally. <https://ollama.com>.
- OpenAI, 2024. GPT-4 technical report. URL: <https://arxiv.org/abs/2303.08774>, arXiv:2303.08774.
- Rane, N., Choudhary, S., Rane, J., 2023. Integrating building information modelling (BIM) with ChatGPT, Bard, and similar generative artificial intelligence in the architecture, engineering, and construction industry: applications, a novel framework, challenges, and future scope. SSRN Electronic Journal doi:10.2139/ssrn.4645601.
- Reimers, N., Gurevych, I., 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-Networks, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics. pp. 3982–3992. URL: <https://arxiv.org/abs/1908.10084>.
- Saka, A., Taiwo, R., Saka, N., Oluleye, B., Akanbi, L., 2024. Integrated BIM and machine learning system for circularity prediction of construction demolition waste. arXiv preprint arXiv:2407.14847 URL: <https://arxiv.org/abs/2407.14847>.
- Sammour, F., Xu, J., Wang, X., Hu, M., Zhang, Z., 2024. Responsible AI in construction safety: Systematic evaluation of large language models and prompt engineering. URL: <https://arxiv.org/abs/2411.08320>, arXiv:2411.08320.
- Samsami, R., 2024. Optimizing the utilization of generative artificial intelligence (AI) in the AEC industry: ChatGPT prompt engineering and design. CivilEng 5, 971–1010. doi:10.3390/civileng5040049.
- Santos, K.T., Sampaio, M.A.B., 2021. Aplicação da metodologia RASE na norma de acessibilidade associada ao BIM. Simpósio Brasileiro de Gestão e Economia da Construção 12, 1–8. doi:10.46421/sibragec.v12i00.500.
- Solihin, W., Eastman, C., 2015. Classification of rules for automated BIM rule checking development. Automation in Construction 53, 69–82. doi:10.1016/j.autcon.2015.03.003.
- Souza, F., Nogueira, R., Lotufo, R., 2020. BERTimbau: Pretrained BERT models for Brazilian Portuguese, in: 9th Brazilian Conference on Intelligent Systems (BRACIS).
- Tan, P.N., Steinbach, M., Kumar, V., 2006. Introduction to Data Mining. 1 ed., Pearson Education, Boston.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., Lample, G., 2023. LLaMA: Open and efficient foundation language models. URL: <https://arxiv.org/abs/2302.13971>, arXiv:2302.13971.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I., 2017. Attention is all you need, in: Advances in Neural Information Processing Systems (NeurIPS). URL: <https://arxiv.org/abs/1706.03762>.
- Zhang, T., Kishore, V., Wu, F., Weinberger, K.Q., Artzi, Y., 2020. BERTScore: Evaluating text generation with BERT, in: Proceedings of the 8th International Conference on Learning Representations (ICLR).

Zhao, W.X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Nie, J.Y., Wen, J.R., 2023. A survey of large language models. arXiv preprint arXiv:2303.18223 URL: <https://arxiv.org/abs/2303.18223>.